

Parallel Port IP Core — Altera DE-Series

(altera_up_avalon_parallel_port)

Caractéristiques

Composant	altera_up_avalon_parallel_port
Largeur	1 à 32 bits (configurable)
Direction	Input / Output / Bidirectionnel
Include	altera_up_avalon_parallel_port.h
Cible	DE-series boards (Qsys)

Directions de port

- Input only — capture uniquement
- Output only — pilotage uniquement
- Bidirectionnel (tristate) — direction bit à bit

Usages typiques

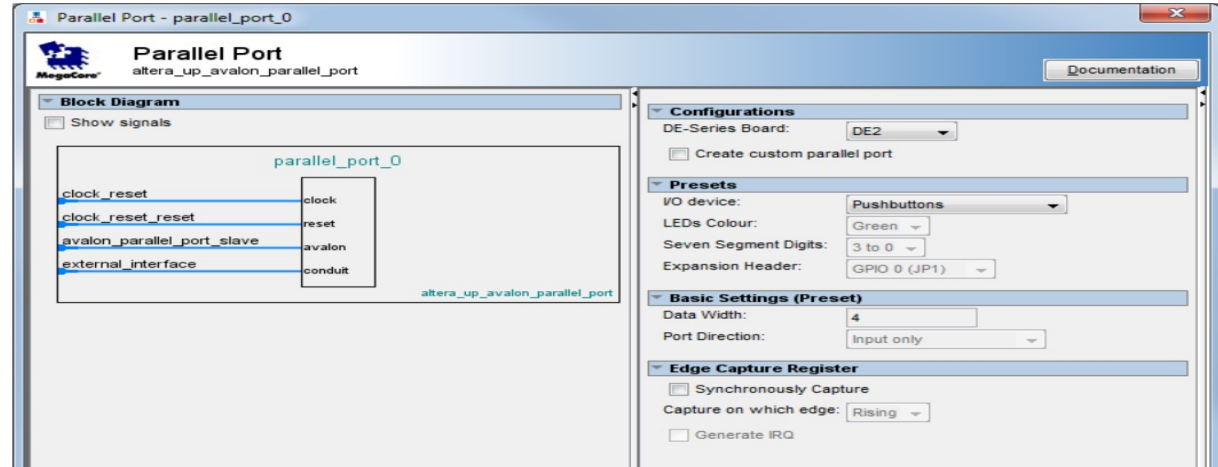
- Contrôle de LEDs
- Lecture de switches/boutons
- Contrôle d'afficheurs 7 segments
- Communication avec GPIO externe
- Interface avec expansion headers

Edge Capture Register

- Synchronously Capture : activable
- Capture sur front montant, descendant ou les deux
- Generate IRQ : IRQ asserté à chaque front capturé

Parallel Port IP Core — (altera_up_avalon_parallel_port)

Modèle de programmation



Offset in bytes	Register name		Read/Write	Bits $(n - 1) \dots 0$
0	data	Input	R	Data value currently on Parallel Port inputs.
		Output	W	New value to drive on Parallel Port outputs.
4	direction		R/W	Individual direction control for each I/O port. A value of 0 sets the direction to input; 1 sets the direction to output.
8	interruptmask		R/W	IRQ enable/disable for each input port. Setting a bit to 1 enables interrupts for the corresponding port.
12	edgecapture		R/W	Edge detection for each input port.

Parallel Port — Modèle de programmation (Register Map)

Offset	R/W	Registre	Description
0	R/W	data	Lecture : valeur sur les entrées. Écriture : valeur à appliquer sur les sorties.
4	R/W	direction	Bidirectionnel uniquement. Bit=0 → entrée, Bit=1 → sortie. Reset : tous à 0 (entrées).
8	R/W	interruptmask	Bit=1 active l'IRQ pour le bit correspondant du registre edgecapture. Reset : tous à 0.
12	R/W	edgecapture	Bit=1 si un front a été détecté sur l'entrée n. Tout write remet à 0. (Parallel Port : lecture seule aussi)

Remarque : les registres direction, interruptmask et edgecapture n'existent que si la configuration matérielle les active. Lire un registre absent retourne une valeur indéfinie.

Registre data

- Lecture → valeur présente sur les entrées
- Écriture → valeur appliquée sur les sorties
- Mode input seul : écriture sans effet

Registre direction (bidirectionnel)

- Bit n = 0 → port n configuré en entrée (haute impédance)
- Bit n = 1 → port n configuré en sortie
- Après reset : tous les bits à 0 (tout en entrée)

Fonctions HAL — altera_up_avalon_parallel_port.h (1/2)

alt_up_parallel_port_open_dev()

```
alt_up_parallel_port_dev* alt_up_parallel_port_open_dev(const char* name)
```

Ouvre le périphérique par son nom Qsys. Ex : "/dev/green_leds". Retourne NULL si non trouvé.

alt_up_parallel_port_read_data()

```
unsigned int alt_up_parallel_port_read_data(alt_up_parallel_port_dev* pp)
```

Lit le registre data. Retourne la valeur présente sur les entrées (ou la dernière valeur écrite sur les sorties).

alt_up_parallel_port_write_data()

```
void alt_up_parallel_port_write_data(alt_up_parallel_port_dev* pp, unsigned data)
```

Écrit dans le registre data. Applique la valeur sur les sorties. Sans effet en mode input seul.

alt_up_parallel_port_read_direction()

```
unsigned int alt_up_parallel_port_read_direction(alt_up_parallel_port_dev* pp)
```

Lit le registre direction. Chaque bit indique si le port correspondant est en entrée (0) ou sortie (1).

Fonctions HAL — altera_up_avalon_parallel_port.h (2/2)

alt_up_parallel_port_set_port_direction()

```
void alt_up_parallel_port_set_port_direction(alt_up_parallel_port_dev* pp, unsigned direction)
```

Définit le registre direction complet. Bit=0 : entrée, Bit=1 : sortie.

alt_up_parallel_port_set_all_bits_to_input/output()

```
void alt_up_parallel_port_set_all_bits_to_input(pp) / set_all_bits_to_output(pp)
```

Configure tous les bits du port en entrée ou en sortie en une seule opération.

alt_up_parallel_port_set_bit_to_input/output()

```
void alt_up_parallel_port_set_bit_to_input(pp, unsigned int bit) / set_bit_to_output(pp, bit)
```

Configure un seul bit en entrée ou en sortie sans modifier les autres bits.

alt_up_parallel_port_set_interrupt_mask()

```
void alt_up_parallel_port_set_interrupt_mask(alt_up_parallel_port_dev* pp, unsigned mask)
```

Active les IRQ pour les bits dont le masque est à 1. Lire le masque courant avec read_interrupt_mask().

alt_up_parallel_port_read_edge_capture()

```
unsigned int alt_up_parallel_port_read_edge_capture(alt_up_parallel_port_dev* pp)
```

Lit le registre edgecapture. Bit=1 si un front a été détecté. La lecture efface automatiquement le registre.

Exemple — Lecture switches → Écriture LEDs

```
#include "altera_up_avalon_parallel_port.h"
#include <stdio.h>

int main() {
    alt_up_parallel_port_dev* leds;
    alt_up_parallel_port_dev* sw;
    unsigned int val;

    // Ouverture des périphériques
    leds = alt_up_parallel_port_open_dev("/dev/leds");
    sw   = alt_up_parallel_port_open_dev("/dev/switches");

    if (!leds || !sw) {
        printf("Erreur: périphérique introuvable\n");
        return -1;
    }

    while (1) {
        // Lecture des switches
        val = alt_up_parallel_port_read_data(sw);
        // Recopie sur les LEDs
        alt_up_parallel_port_write_data(leds, val);
    }
    return 0;
}
```

1

open_dev()

Ouvre chaque périphérique par son nom Qsys. Vérifier NULL avant usage.

2

read_data()

Lit la valeur binaire des switches (registre data en entrée).

3

write_data()

Écrit la valeur sur les LEDs (registre data en sortie).

Exemple — Interruption sur front bouton (Edge Capture + IRQ)

```
#include "altera_up_avalon_parallel_port.h"
#include <sys/alt_irq.h>

alt_up_parallel_port_dev* btn;

// Handler IRQ
void bouton_irq(void* ctx) {
    unsigned int edge;
    // Lire (et effacer) le registre edgecapture
    edge = alt_up_parallel_port_read_edge_capture(btn);
    if (edge & 0x1)
        printf("Bouton 0 appuyé !\n");
}

int main() {
    btn = alt_up_parallel_port_open_dev("/dev/buttons");

    // Activer IRQ sur bit 0
    alt_up_parallel_port_set_interrupt_mask(btn, 0x1);

    // Enregistrer le handler
    alt_irq_register(BUTTONS_IRQ, NULL, bouton_irq);

    while(1); // Attente interruption
    return 0;
}
```

1

set_interrupt_mask()

Active l'IRQ pour les bits souhaités (masque bit à bit).

2

alt_irq_register()

Enregistre le handler IRQ Nios II lié au composant buttons.

3

read_edge_capture()

Dans le handler : lit et efface le registre pour identifier le bit source.

4

Effacement auto

La lecture du registre edgecapture le remet à zéro (Parallel Port).

Récapitulatif — Toutes les fonctions HAL Parallel Port

Fonction	Signature simplifiée	Rôle	Famille
<code>open_dev()</code>	<code>(name) → dev*</code>	Ouvre le périphérique	Init
<code>read_data()</code>	<code>(dev) → uint</code>	Lit les entrées	Data
<code>write_data()</code>	<code>(dev, data)</code>	Pilote les sorties	Data
<code>read_direction()</code>	<code>(dev) → uint</code>	Lit le registre direction	Direction
<code>set_port_direction()</code>	<code>(dev, dir)</code>	Configure toutes les directions	Direction
<code>set_all_bits_to_input()</code>	<code>(dev)</code>	Tout en entrée	Direction
<code>set_all_bits_to_output()</code>	<code>(dev)</code>	Tout en sortie	Direction
<code>set_bit_to_input()</code>	<code>(dev, bit)</code>	Un bit en entrée	Direction
<code>set_bit_to_output()</code>	<code>(dev, bit)</code>	Un bit en sortie	Direction
<code>set_interrupt_mask()</code>	<code>(dev, mask)</code>	Active les IRQ par masque	IRQ
<code>read_interrupt_mask()</code>	<code>(dev) → uint</code>	Lit le masque IRQ courant	IRQ
<code>read_edge_capture()</code>	<code>(dev) → uint</code>	Lit et efface les fronts détectés	IRQ
<code>read_interrupt_pending()</code>	<code>(dev) → uint</code>	Lit les IRQ en attente	IRQ